

Adversarial Verification of the Unchessed UCI Chess Engine Main Branch

Manus AI

Read-Only Verification and Research Facilities

Unchessed-UCI-Engine

Audit branch: `manus/research-facilities`

Abstract—This paper reports a read-only adversarial verification audit of the Unchessed Universal Chess Interface (UCI) chess-engine repository. The target was the exact main branch at commit `818ef9dd5bb7be64fd6085f7c1910b953390da6e`; all dynamic checks used an immutable archive of that tree, and no source code in the target branch was modified. Five independent audit lanes examined build and test infrastructure, core chess correctness, search and evaluation, UCI/runtime behavior, and Python data/security tooling. The audit reproduced twelve high or medium-impact defects and identified additional quality-gate and reproducibility gaps. Core findings include malformed FEN states that can produce illegal special moves, hash corruption, or a panic; mate being lost at the halfmove threshold; premature repetition draws; promotion material omitted from static exchange evaluation; incorrect stalemate scoring in quiescence; and materially violated node/time limits. UCI findings include uncharged adapter analysis, delayed multi-threaded responses, ignored searchmoves, and non-implemented pondering. Tooling findings include duplicate-game train/validation leakage, corrupt partial artifacts, unsafe checkpoint deserialization, shell injection in an environment-derived command, forged provenance metadata, blocking timeout logic, non-atomic shard publication, and unbounded relabeling memory retention. Passing tests and ordinary runtime smokes demonstrate useful baseline coverage but do not establish adversarial robustness, playing strength, or production readiness.

Index Terms—chess engine, adversarial verification, UCI, FEN, NNUE, static exchange evaluation, data provenance, software security, reproducibility

I. INTRODUCTION

A chess engine can pass a conventional unit-test suite while still violating chess rules, protocol contracts, resource limits, or data-integrity guarantees. These failures are especially important at boundaries that ordinary tests rarely exercise: malformed but parse-accepted positions, terminal states, very small time and node budgets, duplicate training inputs, process interruption, and hostile external artifacts. This paper presents an adversarial audit of the Unchessed UCI engine’s main branch and separates what was independently verified from what remains untested.

The audit was requested as a whole-codebase review covering bugs, errors, genuine improvements, stability, and enhancements. To avoid contaminating the target, the target commit was exported as a tar archive and extracted into temporary directories. Independent verification agents inspected different subsystems, created temporary external harnesses where necessary, and returned exact commands, outputs, source

locations, verdicts, and limitations. A synthesis agent was unavailable after the five lanes completed because the session reached its credit boundary; the lane outputs were therefore consolidated manually without rerunning or modifying the target.

The primary conclusion is negative but actionable: ordinary valid-input paths have meaningful coverage, yet the branch is not robust at adversarial input, strict deadline, protocol, crash-integrity, or provenance boundaries. The paper does not claim a playing-strength result. It also does not claim that any recommended remediation has been implemented.

II. TARGET AND VERIFICATION METHOD

A. Exact target

The audit target was main at commit `818ef9dd5bb7be64fd6085f7c1910b953390da6e`, tree `062aa187b4d4eb3e4ff69681878f9a4acd946044`. The immutable archive had SHA-256 digest beginning `3a4a3586` and ending `c017618`; the complete digest is retained in the audit report. The audit branch was `manus/research-facilities`. The target branch, archive, and source files were not edited.

B. Evidence ladder

The audit distinguishes four levels. *Source inspection* establishes the behavior described by code. *Automated test evidence* requires a relevant test or checker to run to completion. *Runtime evidence* requires a compiled binary or external harness to process real input. *End-to-end evidence* requires a realistic process and sufficient repetition to make an outcome meaningful. A reproduced bug can earn runtime evidence without earning a general strength or production-readiness conclusion.

Every finding was reported with a claim, evidence level, exact method, verdict, anomalies, limitations, confidence, and a concrete confidence-raising check. Verdicts use four categories: confirmed, refuted, partially confirmed, and could not verify. A source-level security or resource issue is called confirmed when the unsafe behavior is directly established, even if a full malicious payload or production-scale stress test was intentionally not executed.

C. Independent audit lanes

Five lanes were run against the immutable snapshot:

- 1) build configuration, tests, skips, CI, and reproducibility;
- 2) board, FEN, move generation, make/copy, hashing, repetition, draws, SEE, and memory safety;
- 3) alpha-beta search, pruning, quiescence, limits, TT, NNUE, and SIMD;
- 4) UCI parsing, clocks, options, threads, Lazy SMP, model/book loading, and process behavior;
- 5) Python data, training, relabeling, provenance, subprocesses, timeouts, and security.

III. BASELINE RESULTS AND QUALITY GATES

A. Rust tests

The extracted snapshot compiled with the available modern Rust toolchain. The core command

```
cargo test -p unchessed-core -- --test-threads=1
```

reported 118 passed, zero failed, and six ignored tests. The ignored deep perft test was separately run with `--ignored` and passed one assertion in 13.49 seconds. An independent randomized external harness performed 1,000 legal copy/make/hash checks.

The default result is useful evidence for valid ordinary paths, but it is not exhaustive. The deep perft assertion is not part of the default gate, and the adversarial defects in Sections IV–VI escaped these tests. The test result therefore earns automated-test evidence, not universal chess correctness.

B. Build and release gaps

The documented `scripts/build-and-test.sh` checks for Cargo but not a native linker. With the documented offline Rust toolchain and no `cc`, `gcc`, or `clang`, the gate failed with `error: linker cc not found`. After installing the native compiler in the audit VM, the same gate passed. This is a confirmed setup gap.

The script’s release branch builds optimized binaries but does not run optimized tests. A separate direct command, `cargo test --workspace --release --lib --bins`, passed 118 tests and ignored six. Thus the release-script result is not equivalent to a release test result. No CI configuration was found in the usual repository locations. Formatting reported 146 hunks across 16 files, while strict Clippy reported 37 core-library diagnostics and 52 core-library-test diagnostics under the audited toolchain.

The snapshot’s moving `stable` Rust channel and lower-bound-only Python requirements also weaken reproducibility. The Python suite in the archive form failed because one bracket-check test requires Git metadata: 363 passed, 22 skipped, and one failed. Explicitly supplying 21 Rust files to the checker passed, demonstrating that this particular failure is archive/Git enumeration behavior rather than a bracket imbalance.

TABLE I
BASELINE AUTOMATED EVIDENCE

Check	Result	Boundary
Core Rust tests	118 pass, 6 ignored	Valid paths covered; adversarial gaps remain
Deep perft	1 pass when explicitly enabled	Omitted by default gate
Random legal hash harness	1,000 pass	Does not cover malformed FEN
Python archive suite	363 pass, 22 skip, 1 fail	Git-dependent failure in archive form
Rust bracket checker	21 explicit files pass	Structural only
Formatting	146 hunks fail check	Not runtime evidence
Strict Clippy	37+52 diagnostics	Toolchain-sensitive quality gap

IV. CORE CORRECTNESS FINDINGS

A. Malformed FEN creates illegal state and panic paths

Claim. Parse-accepted FEN states are safe for move generation and make/copy operations.

Evidence and method. Source inspection found that `fen.rs:11--55` does not enforce eight ranks and eight files per rank. Castling and en-pasants fields at lines 63–93 are accepted based primarily on syntax, with king-count validation but no full state coherence check. `movegen.rs:694--731` generates castling from rights, path emptiness, and attack tests without proving that the required king and rook exist. `board.rs:325--400` then assumes those pieces exist.

A temporary external harness compiled against the extracted tree. For `4k3/8/8/8/8/8/8/8/4K3 w K - 0 1`, it reported `castle_legal=["e1g1"]`, followed by incremental hash `939b9ada67fd5b78` and recomputed hash `b22245a5deffe394`. A king-off-home-square position with K rights caused a panic at `board.rs:332:40: make: no piece on from-square`. A malformed en-pasants FEN emitted `e5d6` and produced a hash mismatch after make. Five-rank, seven-file, and empty-rank FENs were also accepted.

Verdict: confirmed. The defect is externally reachable through `uci.rs:831--890` and can cause illegal moves, state corruption, or process termination. It is an availability/correctness defect, not a demonstrated memory-unsafe exploit.

Anomalies and limits. Valid-position perft, valid en-pasants, and 1,000 randomized hash checks passed. The audit did not exhaustively fuzz every malformed FEN. Confidence is high for the reproduced cases; strict six-field/rank/file/state validation and differential FEN fuzzing would raise it.

B. Mate is lost at the halfmove threshold

Claim. Terminal mate takes precedence over a halfmove draw condition.

Evidence and method. At `search.rs:506--512`, the code returns `draw(ply)` for `halfmove >= 100` before later checkmate handling. A known mate-in-one at halfmove 99 produced a child with halfmove 100 and zero legal moves, but depth-one search returned score zero instead of a mate score.

Verdict: confirmed. The tested path scores checkmate at the threshold as a draw. Root and interior behavior is also inconsistent because the root loop can select a resetting capture from a halfmove-100 root while interior nodes draw immediately.

Limits. The intended policy for claimable 50-move versus automatic 75-move handling is not specified by this result. A complete terminal-state matrix and explicit rule-policy specification are needed.

C. Repetition draws occur on the second occurrence

Claim. Repetition detection implements threefold repetition.

Evidence and method. `search.rs:291--315` uses `.any(|h| h == hash)` to detect an earlier occurrence, and the draw path treats one prior matching hash as a draw. A reversible sequence returned to the initial position with one historical occurrence, meaning two total occurrences. The probe reported `score_with_one_prior_occurrence=0` versus `score_without_history=-865`.

Verdict: confirmed. The reproduced path declares a draw on the second occurrence rather than the required third occurrence.

Limits. Root-history bounds and irreversible-move handling were not exhaustively fuzzed. The result is nevertheless direct evidence of a rules mismatch in the tested path.

D. Irrelevant en-passant state fragments hashes

Claim. Position identity distinguishes only legally relevant state.

Evidence and method. `board.rs:308--323` XORs an en-passant file whenever the field is nonempty, without checking for a legal capture. In a position where no black pawn could capture the target, the EP and non-EP forms had identical legal moves but hashes `81607e0d394685b4` and `9e853282a70936ab`, respectively.

Verdict: confirmed. This can fragment TT identity and miss repetition equivalences. A pinned EP opportunity was not exhaustively tested, so the exact normalization boundary remains incomplete.

E. SEE omits promotion material

Claim. Static exchange evaluation includes promotion gain.

Evidence and method. `see.rs:140--154` initializes gain from the captured piece and does not add the promoted-piece-minus-pawn delta. A temporary probe with pawn 82, rook 477, and queen 1025 returned `see_quiet_promotion=0` instead of +943 and `see_capture_promotion=477` instead of +1420.

Verdict: confirmed. Promotion ordering and SEE-based pruning can be wrong. Ordinary capture tests passing does not cover this promotion-specific defect.

V. SEARCH AND EVALUATION FINDINGS

A. Quiescence mis-scores stalemate

Claim. A non-check stalemate qnode receives the draw score.

Evidence and method. At `search.rs:395--404`, non-check qsearch initializes its best score from static evaluation and generates captures only. The return path at lines 416–480 has no non-check/no-legal-move stalemate case. A legal stalemate FEN produced `q_stalemate=(-1245,-1245)`; a checkmate control produced `q_checkmate=(-1256,-30000)`.

Verdict: confirmed. The qsearch leaf can treat stalemate as a positional score rather than a draw. The direct defect is confirmed even where a particular root fixture may not change the selected move.

B. Node and time limits are not hard bounds

Claim. Small go nodes and go movetime requests constrain work closely to the requested values.

Evidence and method. `search.rs:271--288` checks limits only when `nodes & 2047 == 0`; node increments occur earlier in qsearch and negamax. A one-node request completed four iterations and reported approximately 3,173 ms. A separate UCI probe reported 1,088 nodes for go nodes 1. A fresh-process one-millisecond limit returned in 3,188, 3,208, and 3,279 ms because root setup and lazy magic-table initialization occurred before a deadline check.

Verdict: confirmed. The engine materially violates small requested node/time bounds. The five-millisecond minimum in `Limits::budget` further changes the semantics of very short requests.

Limits. The magnitude depends on host, cold/warm state, and configuration. Ordinary long time controls were not exhaustively characterized.

C. NNUE, TT, and SIMD non-findings

The targeted NNUE SIMD scalar-parity test passed on a host advertising AVX2 and FMA. The core tests passed. Inspection of `tt.rs:145--193` found coherent score clamping, mate conversion, and XOR-key logic, and no deterministic false TT hit was reproduced. These are partial confirmations only. No Miri, sanitizer, TSAN, non-AVX2, 32-bit, or long-running concurrent stress run was performed. Nonfinite NNUE payload handling remains a source-level suspicion because f32 values are deserialized without a demonstrated finiteness scan; a malicious full-sized model was not executed.

VI. UCI AND RUNTIME FINDINGS

A. Adapter analysis is outside the move clock

Claim. Adapter responses obey the requested move time.

Evidence and method. At `uci.rs:1171--1267`, pending-opponent analysis can run depth-14/400,000-node and depth-12/250,000-node searches before the actual search clock is created at `search.rs:915--919`. With position `startpos moves a2a3 and go movetime 5`, a real adapter returned after approximately 664.8 ms and logged opponent analysis.

Verdict: confirmed. The adapter can spend substantial uncharged time before its timed search.

B. Lazy SMP delays bestmove after deadline

Claim. Multi-threaded searches return promptly at the requested deadline.

Evidence and method. `uci.rs:1388--1431` scopes helper threads and cannot print `bestmove` until they join. With 64 accepted threads and `go movetime 5`, real trials returned after 37.4, 58.9, 34.8, and 54.7 ms.

Verdict: confirmed. The response deadline is materially exceeded. Absolute latency is host-dependent, but the violation is repeatable.

C. Restricted root search is ignored

Claim. `searchmoves` restricts root move selection.

Evidence and method. `uci.rs:892--920` has no `searchmoves` branch. The real Reviewer binary received `go depth 1 searchmoves e2e4` and returned `bestmove d2d4`.

Verdict: confirmed. The protocol command is silently ignored and can return an unauthorized move.

D. Pondering is not implemented

Claim. Ponder commands follow the UCI ponder lifecycle.

Evidence and method. No `ponder` or `ponderhit` dispatch exists in the inspected parser and command loop. A real binary given `go ponder movetime 50` returned `bestmove blc3` in 27.9 ms without `ponderhit` or `stop`.

Verdict: confirmed. Ponder is not implemented; the engine behaves as an ordinary searcher and emits a move immediately.

VII. DATA INTEGRITY AND SECURITY FINDINGS

A. Duplicate games leak across v5 splits

Claim. The v5 builder's game-disjoint split prevents duplicate games from crossing train and validation.

Evidence and method. `pretrain_v5_data.py:342--343` derives game identity from source `basename` plus `Round/ordinal`. Two identical PGNs under different filenames therefore receive different hashes. A hermetic reproducer with `--val-games 1` returned success, empty hash intersection, but `ROWWISE_SEMANTIC_EQUAL=True` and identical semantic digest beginning `6319c99c`; the complete digest is retained in the audit report.

Verdict: confirmed. The claimed game-disjoint property fails for renamed duplicate inputs.

B. Malformed rows leave corrupt partial datasets

Claim. Invalid rows fail cleanly without corrupting published output.

Evidence and method. `pretrain_move_dataset.py:186--198` preallocates arrays; lines 200–229 do not maintain valid-row indices; lines 301–306 remove a prefix based on the number of bad rows rather than actual bad positions. A five-row input with an invalid middle row returned code 1 but left a training shard with game IDs `[10, 32587, 12]`, including an uninitialized value, and no manifest.

Verdict: confirmed. Invalid middle rows can corrupt data, discard valid records, and leave partial artifacts.

C. Unsafe PyTorch checkpoint deserialization

Claim. Caller-selected checkpoints are safely loaded.

Evidence and method. Multiple paths use `torch.load(..., weights_only=False)`, including `export_unarchitected_v1.py:37`, `pretrain_v1_a100.py:482--483`, `train_nnue_xt_a100.py:402`, and several Unarchitected training paths. The option accepts pickle-capable objects, so an attacker who controls a selected checkpoint can execute code during loading.

Verdict: confirmed conditional security defect. Torch was unavailable, so no payload was executed. The unsafe API and caller-controlled path are sufficient to establish the code-level risk; deployment exposure remains environment-dependent.

D. Environment-derived shell injection

Claim. Cloud engine fetching cannot execute arbitrary environment-derived text.

Evidence and method. `maia3_cloud_selfplay/generate.py:1252--1255` interpolates `SF_ARCH` and virtual-environment hints into a command string, and `_run` at 1196–1206 calls it with `shell=True`. A temporary environment value containing `; touch /tmp/.../injected # created the marker file in a hermetic reproducer`.

Verdict: confirmed conditional command-injection defect. The attack requires control over the build environment or template input and runs with the account's privileges. No network access or privilege escalation was used in the reproduction.

E. Crash-unsafe and unverifiable data publication

Several data-path integrity defects were confirmed. v5 shards are opened under their final name before the header is complete; a temporary writer observed a visible final-named shard with an all-zero header and invalid magic before close. The teacher worker accepts a caller-supplied `--input-sha256` and records it without comparing it to the actual input; a false digest was accepted with exit code 0. Depth/time calibration uses blocking `readline()` inside deadline loops; a silent fake engine caused an outer three-second timeout to expire despite an internal 0.1-second timeout. NNUE relabeling reads complete sidecars and records into memory before output, establishing an unbounded full-input retention pattern. v5 manifests record source paths and generated-output hashes but not immutable source hashes or revisions.

VIII. STABILITY AND IMPROVEMENT ASSESSMENT

A. What is stable within the tested boundary

The audited branch has meaningful baseline stability on ordinary valid paths. Core tests passed, deep perf test passed when explicitly requested, randomized valid copy/make/hash checks passed, SIMD scalar parity passed on the available AVX2/FMA host, and ordinary UCI smoke tests completed. TensorPackage structural/CRC checks and malformed Polyglot-length handling

were present. No Rust memory-unsafe undefined behavior was confirmed, and no deterministic TT false-hit defect was reproduced.

B. Where stability breaks

The same branch is not stable at malformed-input, terminal-rule, hard-limit, protocol, crash-publication, or hostile-artifact boundaries. The reproduced FEN panic is an availability defect. The terminal and SEE defects can alter move ordering or scores. Limit defects can violate GUI or caller assumptions. Data defects can poison evaluation splits or publish misleading artifacts. Security defects can execute attacker-controlled code when the relevant inputs are untrusted.

The proper summary is therefore:

Ordinary valid-input paths have meaningful regression coverage, but the branch is not robust at adversarial input, strict resource-bound, protocol-compliance, or crash-integrity boundaries.

C. Evidence-supported improvements

The following improvements are directly supported by reproduced behavior, but were not implemented in this audit:

- 1) enforce strict FEN grammar and coherent castling/en-passant state, with defensive special-move checks;
- 2) evaluate checkmate and stalemate before draw conditions and specify claimable versus automatic draw policy;
- 3) require the correct repetition count and normalize irrelevant en-passant in repetition identity;
- 4) add promotion deltas to SEE and promotion-specific regression fixtures;
- 5) enforce node/time limits at appropriate boundaries, including root setup, adapter pre-search, and helper cancellation;
- 6) implement or explicitly reject unsupported UCI commands, especially `searchmoves` and `ponder`;
- 7) canonicalize game identity, verify metadata hashes, track valid rows explicitly, and publish data through temporary files plus atomic rename;
- 8) use safe checkpoint loading and argument-vector subprocesses instead of shell-interpolated commands;
- 9) add exact toolchain/dependency metadata, release tests, deep perft, formatting, lint, and CI enforcement.

IX. NOT VERIFIED AND OUT OF SCOPE

The audit did not establish exhaustive legal correctness, differential agreement over a large randomized corpus, or absence of all malformed-input failures. It did not run Miri, AddressSanitizer, UndefinedBehaviorSanitizer, ThreadSanitizer, Valgrind, 32-bit builds, Windows/macOS/ARM builds, non-AVX2 builds, or long-duration concurrency stress. It did not execute a malicious full-sized PyTorch payload, production-scale OOM test, or every hostile UCI/model/book/package corpus. GPU, Torch, ONNX, A100, cloud, and production-scale training were not verified. A complete GUI UCI conformance test and fixed-suite NPS regression were not run.

No new human-versus-engine game, statistically powered engine match, or SPRT decision for `main` was run as part

of this audit. No Elo, playing-strength, tactical-strength, or default-change conclusion follows. Historical or bounded game records would not substitute for a provenance-complete paired-game campaign. Finally, no remediation was applied; this paper records the audited state rather than certifying a repaired state.

X. CONCLUSION

A read-only adversarial audit of Unchessed `main` found strong ordinary-path test evidence but multiple confirmed failures at the boundaries most likely to matter for correctness, interoperability, resource control, and training integrity. The most severe engine defects affect malformed FEN safety, terminal scoring, qsearch stalemate handling, SEE promotion accounting, and exact time/node contracts. The most severe tooling defects affect data leakage, partial publication, unsafe checkpoint loading, shell execution, provenance, and timeout enforcement.

The findings should be treated as a verification stop signal for any claim of production readiness or game-facing default promotion. The branch needs strict input validation, explicit rule semantics, hard resource-bound tests, safe artifact publication, and security-hardened tooling before passing from ordinary regression confidence to adversarial robustness. This conclusion is about software behavior and evidence quality; it is not a statement about the engine's Elo.

REFERENCES

- [1] Unchessed AI, "Adversarial verification audit of main," project report, 2026.
- [2] S. Meyer-Kahlen, "Universal Chess Interface protocol," protocol specification. [Online]. Available: <https://www.shredderchess.com/chess-info/features/uci-universal-chess-interface.html>
- [3] FIDE, "Laws of chess," Articles 9.6 and 9.7, official rules. [Online]. Available: <https://handbook.fide.com/chapter/E012023>
- [4] Stockfish Developers, "Stockfish," official chess-engine repository. [Online]. Available: <https://github.com/official-stockfish/Stockfish>
- [5] Stockfish Developers, "Fishtest," distributed testing documentation. [Online]. Available: <https://official-stockfish.github.io/docs/fishtest/>
- [6] PyTorch Developers, "Serialization semantics," official documentation. [Online]. Available: <https://pytorch.org/docs/stable/notes/serialization.html>
- [7] Ronald de Man, "Syzygy endgame tablebases," technical documentation. [Online]. Available: <https://syzygy-tables.info/>